

Exposing Relay over MCP

How the `/api/mcp` endpoint was built, what it exposes, and what it deliberately does not.

A walkthrough for someone who needs to explain this on a call, not just ship it.

1. What MCP is, in one paragraph

Model Context Protocol is a standard way for an AI application to discover and call tools that live somewhere else. Before it, every integration was bespoke: if you wanted Claude Desktop to read your orders, someone wrote a Claude-specific plugin, and then a Cursor-specific one, and then another. MCP replaces that with one contract — a server advertises its tools, any compliant client can call them.

💡 "MCP is a standard plug. We expose our tools once, and any AI tool that speaks it can use them — Claude Desktop, Cursor, or an agent someone builds next year."

2. Why we exposed Relay this way

Relay already had a working tool layer: nine functions with schemas, validation and auditing, wired to a live database. Those tools were only reachable through Relay's own chat.

Exposing them over MCP costs about forty lines and changes the offer:

- The client keeps working in the tool they already use. No new tab, no new login.
- The tools stop being a feature of our product and become **their** capability.
- Any future MCP-speaking client works without us shipping anything.

The commercial version of that sentence:

💡 "You don't have to move into our interface. We can put these tools inside the one your team already has open."

3. The design decision that matters

The tool layer has eight reads and one write. **Only the reads are exposed.**

`issue_refund` is absent, and not because it was hard. The human approval gate lives inside Relay's interface — the card that shows the amount and waits for someone to press approve. If the refund tool were reachable over MCP, an external client could call it directly and the gate would never render. The safety property would be gone, and nothing in the protocol would have stopped it.

So the boundary is drawn by what exists at the edge, not by configuration:

Where	Reads	Write
Relay's own chat	yes	yes, behind the approval gate
Specialists (sub-agents)	yes	no — never delegated
MCP endpoint	yes	no — never exposed

Three different places, one rule: **the write only exists where the gate is.**

💡 "Reads are safe to federate. The write isn't, so it stays where the human approval lives. That's not a setting — the tool simply isn't there."

4. How it was built

One route file. The tools already existed, so the work was adapting them to the protocol.

4.1 Reuse the existing tools

The tool definitions are already `{ description, inputSchema, execute }`. The MCP server registers each one:

```
// src/app/api/mcp/route.ts
import { createMcpHandler } from "mcp-handler";
import { createBusinessTools, createKnowledgeTools } from "@lib/tools";

// Read tools only. The write path is intentionally absent.
const EXPOSED = { ...createBusinessTools(SESSION), ...createKnowledgeTools(SESSION) };
```

Note what is **not** imported: `createWriteTools`. That single omission is the security boundary. It is one line, it is greppable, and a reviewer can verify it in five seconds — which is exactly the property you want a safety control to have.

4.2 Register each tool

```
const handler = createMcpHandler((server) => {
  for (const [name, definition] of Object.entries(EXPOSED)) {
    server.registerTool(
      name,
      { description: tool.description, inputSchema: tool.inputSchema },
      async (input) => {
        const result = await tool.execute(input, {});
        return {
          content: [
            { type: "text", text: result?.summary ?? "No result." },
            { type: "text", text: JSON.stringify(result, null, 2) },
          ],
        };
      },
    );
  }
});
```

Two content blocks on purpose: the one-line summary first, so a client that only renders text still shows a human something readable, then the full structured payload for a client that can use it.

4.3 Instructions for the connecting model

An MCP server can ship instructions that the connecting client passes to its model. Relay's carry the same grounding rule the internal agents have:

"Every fact you report must come from a tool result — do not infer or invent order numbers, amounts or policy rules. Write actions are not available here by design."

Without this, a model connected from outside has none of Relay's prompt discipline and will happily fill gaps.

4.4 Export as HTTP methods

```
export { handler as GET, handler as POST, handler as DELETE };
```

MCP over streamable HTTP uses all three: POST for calls, GET for the event stream, DELETE to end a session.

5. Connecting a client

```
// claude_desktop_config.json
{
  "mcpServers": {
    "relay": { "url": "http://localhost:3000/api/mcp" }
  }
}
```

Restart the client and the tools appear.

6. Verifying it, without a client

Two curl calls prove the endpoint works. Useful on a call when you do not want to restart someone's app.

List the tools:

```
curl -s -X POST http://localhost:3000/api/mcp \
-H 'content-type: application/json' \
-H 'accept: application/json, text/event-stream' \
-d '{"jsonrpc":"2.0","id":1,"method":"tools/list"}'
```

Returns the eight read tools:

```
find_delayed_orders · get_customer · get_operations_summary · get_order  
search_knowledge · search_orders · search_tickets · track_shipment
```

Call one:

```
curl -s -X POST http://localhost:3000/api/mcp \  
-H 'content-type: application/json' \  
-H 'accept: application/json, text/event-stream' \  
-d '{"jsonrpc":"2.0","id":2,"method":"tools/call",  
  "params":{"name":"get_order","arguments":{"orderNumber":"HP-1042"}}}'
```

Returns:

```
"HP-1042 – fulfilled, $537.00, Danielle Okafor"
```

The check that matters — confirm the write tool is not reachable:

```
curl -s -X POST http://localhost:3000/api/mcp \  
-H 'content-type: application/json' \  
-H 'accept: application/json, text/event-stream' \  
-d '{"jsonrpc":"2.0","id":1,"method":"tools/list"}' | grep -c issue_refund  
# → 0
```

Run that one in front of a security-minded client. A demonstrated absence is worth more than a paragraph claiming it.

7. What we did not build, and why

No OpenAPI-to-MCP generation. Turning an arbitrary API spec into a generated MCP server is a different product with a much larger surface: auth brokering, credential custody, versioning, publishing. Relay exposes its own tools, which it already validates and audits.

No authentication yet. The endpoint is open, which is correct for a demo on localhost and wrong for production. `mcp-handler` ships with `withMcpAuth` for bearer-token verification; a real deployment wraps the handler in it and scopes tokens per tenant. Say this plainly if a client asks — claiming a demo is production-hardened is the fastest way to lose the room.

No write tools, ever, without a gate. If a client asks for write access over MCP, the answer is not "we'll expose it" — it is that the approval step has to move with it. Somewhere a human still presses the button.

8. Where to look in the code

File	What it holds
<code>src/app/api/mcp/route.ts</code>	The whole MCP server, about forty lines
<code>src/lib/tools.ts</code>	The tool definitions, shared with the agents
<code>src/lib/audit.ts</code>	Recording — MCP calls are audited like any other

Calls arriving over MCP land in the same audit log as calls from the chat, under the session id `mcp`. Open the Audit tab after a connected client uses a tool and it is there.